

Producto punto máximo (top-1)

Eduardo Alanis García

Table of contents

0.1	Introducción	1
1	Actividad: Producto punto máximo (top-1)	2
1.1	Planteamiento del problema	2
1.2	Analizando el problema	2
1.3	Desarrollo de las actividades.	3
1.3.1	a. Cree el algoritmo A1 con operaciones matriciales explícitas.	3
1.3.2	b. Adapte el algoritmo <i>getmaxdot</i> de las notas (Cap. 3) para el mismo lenguaje de tu implementación para resolver el mismo problema, llámémoslo A2	4
1.3.3	Comprobación con un ejemplo pequeño	6
1.3.4	Consideraciones	6
1.3.5	Datos aleatorios en la esfera unitaria	7
1.3.6	Analice la memoria necesaria por cada algoritmo (A1 y A2) para cada combinación de argumentos; grafique y discuta.	8
1.3.7	Gráfica para $d = 16$	12
1.3.8	Gráfica para $d = 32$	13
1.3.9	Reporte los costos en tiempo real para $m = 10^3, n = 10^6$ y $d = 16$ para cada A1 y A2	15
1.3.10	Grafique la distribución del arreglo <i>dots</i> para la corrida previa, i.e., $m = 10^3, n = 10^6$ y $d = 16$	16
1.4	Conclusiones	17
1.5	Referencias	18

0.1 Introducción

En esta reporte se resuelve la actividad Producto punto máximo (top-1). Dadas las matrices Q y X , el objetivo es asociar cada columna de Q con la columna de X cuyo producto punto sea mayor. Para ello, se implementarán dos algoritmos: el primer algoritmo se desarrollará

mediante operaciones matriciales explícitas, mientras que el segundo será una adaptación del algoritmo `getmaxdot`, presentado en las notas del capítulo 3.

1 Actividad: Producto punto máximo (top-1)

1.1 Planteamiento del problema

Dada la matriz Q de tamaño $d \times m$ y la matriz X de tamaño $d \times n$, para cada vector columna $q \in Q$, encontrar el vector columna $x \in X$ que maximiza su producto punto. Dicho de otra forma, el problema consiste en asociar cada q con $\arg \max\{q \cdot x \mid x \in X\}$. Este será el problema de *encontrar el producto punto máximo* o **top-1**. Obtenga un arreglo de identificadores de columna como resultado (*nns*) y otro con el valor del producto punto máximo (*dots*) para cada columna en Q .

$$Q = \begin{pmatrix} q_{1,1} & q_{1,2} & \cdots & q_{1,m} \\ q_{2,1} & q_{2,2} & \cdots & q_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ q_{d,1} & q_{d,2} & \cdots & q_{d,m} \end{pmatrix}$$

$$X = \begin{pmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{d,1} & x_{d,2} & \cdots & x_{d,n} \end{pmatrix}$$

1.2 Analizando el problema

Sea Q una matriz de $d \times m$ y X una matriz de $d \times n$. Cada columna de Q es una consulta y cada columna de X un candidato. Para la consulta i y el candidato j , el producto punto es:

$$s_{i,j} = \sum_{k=1}^d Q_{k,i} X_{k,j}.$$

El resultado consiste en dos arreglos de longitud m (numero de columnas de la matriz Q):

- $nns[i]$: el identificador de la columna de X que obtiene el mayor producto punto con la columna i de Q .
- $dots[i]$: el valor de ese producto punto máximo.

1.3 Desarrollo de las actividades.

1.3.1 a. Cree el algoritmo A1 con operaciones matriciales explícitas.

Se calcula $Q^T X$. La matriz resultante tiene m filas y n columnas; cada elemento contiene un producto punto. Después, para cada fila, se recorre la fila para encontrar el valor máximo y la columna donde aparece. (Se usó la lógica de la multiplicación de matrices). La multiplicación de matrices permite organizar los productos punto entre todos los vectores de consulta y los candidatos en una sola operación matricial (Golub & Van Loan, 2013).

```
function top1_A1(Q::Matrix{Float32}, X::Matrix{Float32})
    d, m = size(Q)
    dX, n = size(X)

    @assert d == dX
    @assert m > 0 && n > 0

    # Cada elemento tiene el producto punto de la columna i de Q con la columna j de X
    puntos = transpose(Q) * X

    nns = Vector{Int}(undef, m)
    dots = Vector{Float32}(undef, m)

    for i in 1:m # Nos movemos por cada fila de la matriz
        mejor_pos = 1
        mejor_punto = puntos[i, 1]

        for j in 2:n # Nos movemos por cada columna de la fila i
            if puntos[i, j] > mejor_punto
                mejor_pos = j
                mejor_punto = puntos[i, j]
            end
        end

        nns[i] = mejor_pos
        dots[i] = mejor_punto
    end

    return nns, dots
end
```

top1_A1 (generic function with 1 method)

1.3.2 b. Adapte el algoritmo *getmaxdot* de las notas (Cap. 3) para el mismo lenguaje de tu implementación para resolver el mismo problema, llamémoslo A2.

Algoritmo original

```
function mydot(u, x)
    s = 0f0
    for i in eachindex(u, x)
        s += u[i] * x[i]
    end
    s
end

function getmaxdot(u::Vector, X::Matrix)
    maxpos = 1
    # en la siguiente linea, @view nos permite controlar que
    # no se copien los arreglos, y en su lugar, se usen referencias
    maxdot = mydot(u, @view X[:, 1])
    # obtiene el número de columnas e itera a partir del 2do índice
    mfilas, ncols = size(X)
    for i in 2:ncols
        d = mydot(u, @view X[:, i])
        if d > maxdot
            maxpos = i
            maxdot = d
        end
    end

    (maxpos, maxdot)
end

getmaxdot(rand(Float32, 4), rand(Float32, 4, 1000))
```

(281, 2.04931f0)

```
function mydot(u, x)
    s = 0f0

    for i in eachindex(u, x)
        s += u[i] * x[i]
    end
```

```

        return s
    end

function getmaxdot(u, X)
    maxpos = 1
    maxdot = mydot(u, @view(X[:, 1]))

    dfilas, ncols = size(X)
    @assert length(u) == dfilas

    for j in 2:ncols
        d = mydot(u, @view(X[:, j]))

        if d > maxdot
            maxpos = j
            maxdot = d
        end
    end

    return maxpos, maxdot
end

function top1_A2(Q::Matrix{Float32}, X::Matrix{Float32})
    d, m = size(Q)
    dX, n = size(X)

    @assert d == dX
    @assert m > 0 && n > 0

    nns = Vector{Int}(undef, m)
    dots = Vector{Float32}(undef, m)

    for i in 1:m
        nns[i], dots[i] = getmaxdot(@view(Q[:, i]), X)
    end

    return nns, dots
end

```

top1_A2 (generic function with 1 method)

El algoritmo A2 se basa en la implementación de getmaxdot presentada por Téllez (s. f.-a), la

cual se adaptó al problema.

1.3.3 Comprobación con un ejemplo pequeño

```
Q_prueba = Float32[
    1 0
    0 1
]

X_prueba = Float32[
    1 0 -1
    0 2  0
]

nns_prueba_A1, dots_prueba_A1 = top1_A1(Q_prueba, X_prueba)
nns_prueba_A2, dots_prueba_A2 = top1_A2(Q_prueba, X_prueba)

println("A1: nns = ", nns_prueba_A1, ", dots = ", dots_prueba_A1)
println("A2: nns = ", nns_prueba_A2, ", dots = ", dots_prueba_A2)
```

```
A1: nns = [1, 2], dots = Float32[1.0, 2.0]
```

```
A2: nns = [1, 2], dots = Float32[1.0, 2.0]
```

En este ejemplo, ambos algoritmos devuelven los mismos índices y productos máximos: $nns = [1, 2]$ y $dots = [1.0, 2.0]$. Esto confirma que las dos implementaciones resuelven correctamente este caso pequeño ejemplo.

1.3.4 Consideraciones

1. Considere un conjunto de datos como una tupla Q, X .
2. Considere las combinaciones de $m \in \{10^3, 10^6, 10^9\}$, $n \in \{10^3, 10^6, 10^9\}$, $d \in \{8, 16, 32\}$.
3. Las matrices Q y X deberán ser aleatorias cuyos vectores columna estén en la esfera unitaria.
4. Analice la memoria necesaria por cada algoritmo (**A1** y **A2**) para cada combinación de argumentos; grafique y discuta.
5. Reporte los costos en tiempo real para $m = 10^3, n = 10^6$ y $d = 16$ para cada **A1** y **A2**.
6. Grafique la distribución del arreglo $dots$ para la corrida previa, i.e., $m = 10^3, n = 10^6$ y $d = 16$.

1.3.5 Datos aleatorios en la esfera unitaria

Cada matriz se forma con columnas de valores aleatorios. Posteriormente, cada columna se divide entre su norma euclidiana para garantizar que su longitud sea igual a 1. Si se genera una columna cuyos elementos son todos cero, esta se sustituye por el primer vector de la base canónica, que también tiene norma 1. De esta manera, se evita la división por cero y se garantiza que todas las columnas tengan norma unitaria.

```
using Random
using BenchmarkTools
using DataFrames
using Plots
using Statistics

Random.seed!(1234)

function matriz_esfera(d, k)
    M = 2f0 .* rand(Float32, d, k) .- 1f0 # Valores aleatorios en [-1, 1], por eso se multiplica por 2
    for j in 1:k
        suma = 0f0
        for i in 1:d
            suma += M[i, j]^2
        end
        norma = sqrt(suma)
        if norma == 0f0
            M[1, j] = 1f0
        else
            for i in 1:d
                M[i, j] = M[i, j] / norma
            end
        end
    end
    return M
end

function crear_datos(d, m, n)
    Q = matriz_esfera(d, m)
    X = matriz_esfera(d, n)
```

```

# El conjunto de datos se representa como la tupla
return (Q = Q, X = X)
end

```

crear_datos (generic function with 1 method)

Para generar las matrices de prueba se utiliza el módulo Random de Julia, que proporciona funciones para la generación de números aleatorios (Julia contributors, s. f.).

1.3.6 Analice la memoria necesaria por cada algoritmo (A1 y A2) para cada combinación de argumentos; grafique y discuta.

Como se explica en el capítulo 1, `sizeof` permite conocer el tamaño en bytes de los datos almacenados. Con esta información y conociendo el tamaño de las matrices y el funcionamiento de cada algoritmo, podemos estimar cuánta memoria necesita cada uno para ejecutarse. El análisis de la complejidad espacial estudia la cantidad máxima de memoria que utiliza un algoritmo en función del tamaño de su entrada (Sipser, 2013).

- **Entrada común:** Q y X son matrices de tamaño $d \times m$ y $d \times n$, respectivamente. Por eso la memoria de entrada es proporcional a $d(m+n)$.
- **Salidas comunes:** Las salidas son dos arreglos de tamaño m : `nns` y `dots`. Cada elemento de `nns` es un índice `Int` y cada elemento de `dots` es un `Float32`. Por eso la memoria de salida es proporcional a $m(\text{Int} + \text{Float32})$.
- **Memoria adicional de A1:** Al necesitar almacenar la matriz de productos, A1 requiere espacio adicional proporcional a $m \cdot n$. Además, necesita espacio para las salidas.
- **Memoria adicional de A2:** A2 solo necesita espacio para las salidas, porque calcula un producto punto a la vez y conserva solo el máximo actual.

```

tamaños_m = [10^3, 10^6, 10^9]
tamaños_n = [10^3, 10^6, 10^9]
dimensiones = [8, 16, 32]

function bytes_entrada(d, m, n)
    return sizeof(Float32) * d * (m + n)
end

function bytes_salida(m)
    return sizeof(Int) * m + sizeof(Float32) * m
end

```

```

function bytes_totales_A1(d, m, n)
    bytes_productos = sizeof(Float32) * m * n
    return bytes_entrada(d, m, n) + bytes_productos + bytes_salida(m)
end

function bytes_totales_A2(d, m, n)
    return bytes_entrada(d, m, n) + bytes_salida(m)
end

tabla_memoria = DataFrame(
    d = Int[],
    m = Int[],
    n = Int[],
    entradas_GB = Float64[],
    A1_temporal_GB = Float64[],
    A1_total_GB = Float64[],
    A2_total_GB = Float64[]
)

for d in dimensiones
    for m in tamaños_m
        for n in tamaños_n
            temporal_A1 = sizeof(Float32) * m * n

            push!(tabla_memoria, (
                d = d,
                m = m,
                n = n,
                entradas_GB = bytes_entrada(d, m, n) / 10^9,
                A1_temporal_GB = temporal_A1 / 10^9,
                A1_total_GB = bytes_totales_A1(d, m, n) / 10^9,
                A2_total_GB = bytes_totales_A2(d, m, n) / 10^9
            ))
        end
    end
end

tabla_memoria

```

	d	m	n	entradas_GB	A1_temporal_GB	A1_total_GB	A2_total_GB
	Int64	Int64	Int64	Float64	Float64	Float64	Float64
1	8	1000	1000	6.4e-5	0.004	0.004076	7.6e-5
2	8	1000	1000000	0.032032	4.0	4.03204	0.032044
3	8	1000	1000000000	32.0	4000.0	4032.0	32.0
4	8	1000000	1000	0.032032	4.0	4.04403	0.044032
5	8	1000000	1000000	0.064	4000.0	4000.08	0.076
6	8	1000000	1000000000	32.032	4.0e6	4.00003e6	32.044
7	8	1000000000	1000	32.0	4000.0	4044.0	44.0
8	8	1000000000	1000000	32.032	4.0e6	4.00004e6	44.032
9	8	1000000000	1000000000	64.0	4.0e9	4.0e9	76.0
10	16	1000	1000	0.000128	0.004	0.00414	0.00014
11	16	1000	1000000	0.064064	4.0	4.06408	0.064076
12	16	1000	1000000000	64.0001	4000.0	4064.0	64.0001
13	16	1000000	1000	0.064064	4.0	4.07606	0.076064
14	16	1000000	1000000	0.128	4000.0	4000.14	0.14
15	16	1000000	1000000000	64.064	4.0e6	4.00006e6	64.076
16	16	1000000000	1000	64.0001	4000.0	4076.0	76.0001
17	16	1000000000	1000000	64.064	4.0e6	4.00008e6	76.064
18	16	1000000000	1000000000	128.0	4.0e9	4.0e9	140.0
19	32	1000	1000	0.000256	0.004	0.004268	0.000268
20	32	1000	1000000	0.128128	4.0	4.12814	0.12814
21	32	1000	1000000000	128.0	4000.0	4128.0	128.0
22	32	1000000	1000	0.128128	4.0	4.14013	0.140128
23	32	1000000	1000000	0.256	4000.0	4000.27	0.268
24	32	1000000	1000000000	128.128	4.0e6	4.00013e6	128.14
25	32	1000000000	1000	128.0	4000.0	4140.0	140.0
26	32	1000000000	1000000	128.128	4.0e6	4.00014e6	140.128
27	32	1000000000	1000000000	256.0	4.0e9	4.0e9	268.0

1.3.6.1 Graficas

```
function graficar_memoria(d)
    A1 = [bytes_totales_A1(d, m, n) / 10^9 for n in tamaños_n, m in tamaños_m]
    A2 = [bytes_totales_A2(d, m, n) / 10^9 for n in tamaños_n, m in tamaños_m]

    # permutedims convierte el vector en una matriz fila
    etiquetas_A1 = permutedims(["A1, m = $(m)" for m in tamaños_m])
    etiquetas_A2 = permutedims(["A2, m = $(m)" for m in tamaños_m])

    p = plot(
        tamaños_n,
```

```

        A1,
        label = etiquetas_A1,
        xscale = :log10,
        yscale = :log10,
        xlabel = "Número de candidatos, n",
        ylabel = "Memoria total (GB decimales)",
        title = "Memoria estimada para d = $(d)",
        marker = :circle,
        linewidth = 2
    )

    plot!(
        p,
        tamaños_n,
        A2,
        label = etiquetas_A2,
        marker = :diamond,
        linewidth = 2
    )

    return p
end

```

graficar_memoria (generic function with 1 method)

1.3.6.1.1 Gráfica para $d = 8$

```

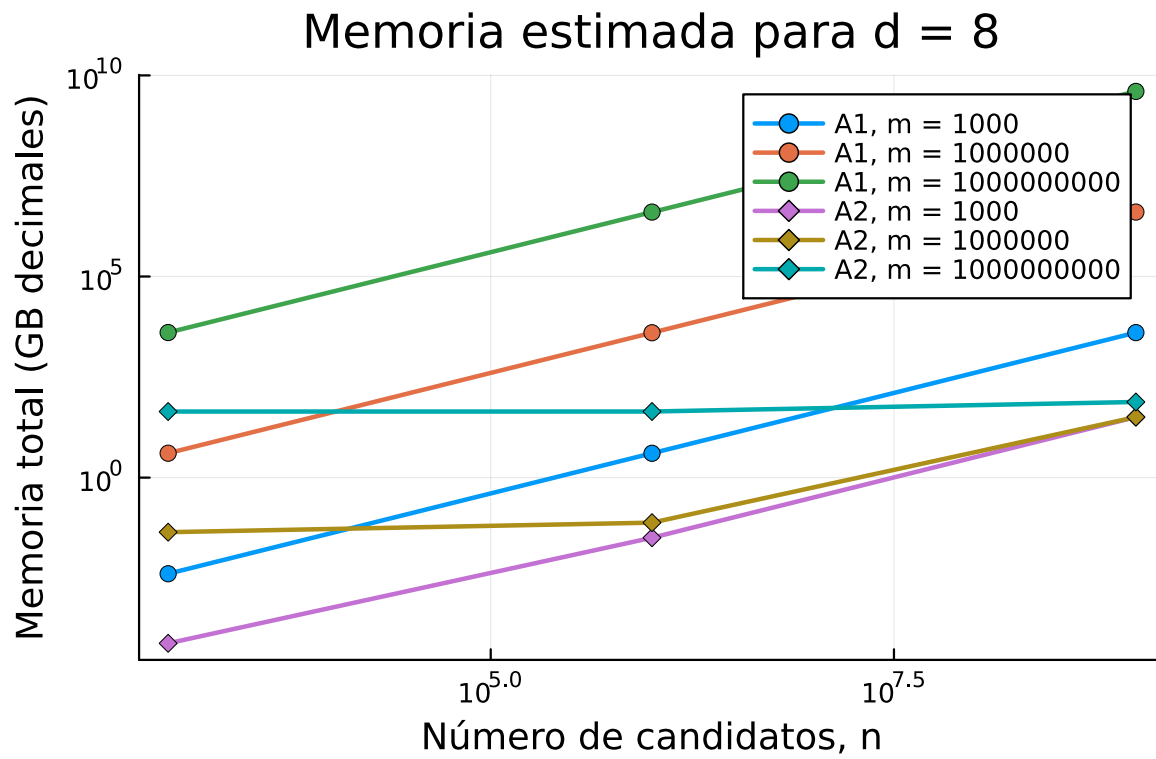
gr()
default(fmt = :png)

```

```

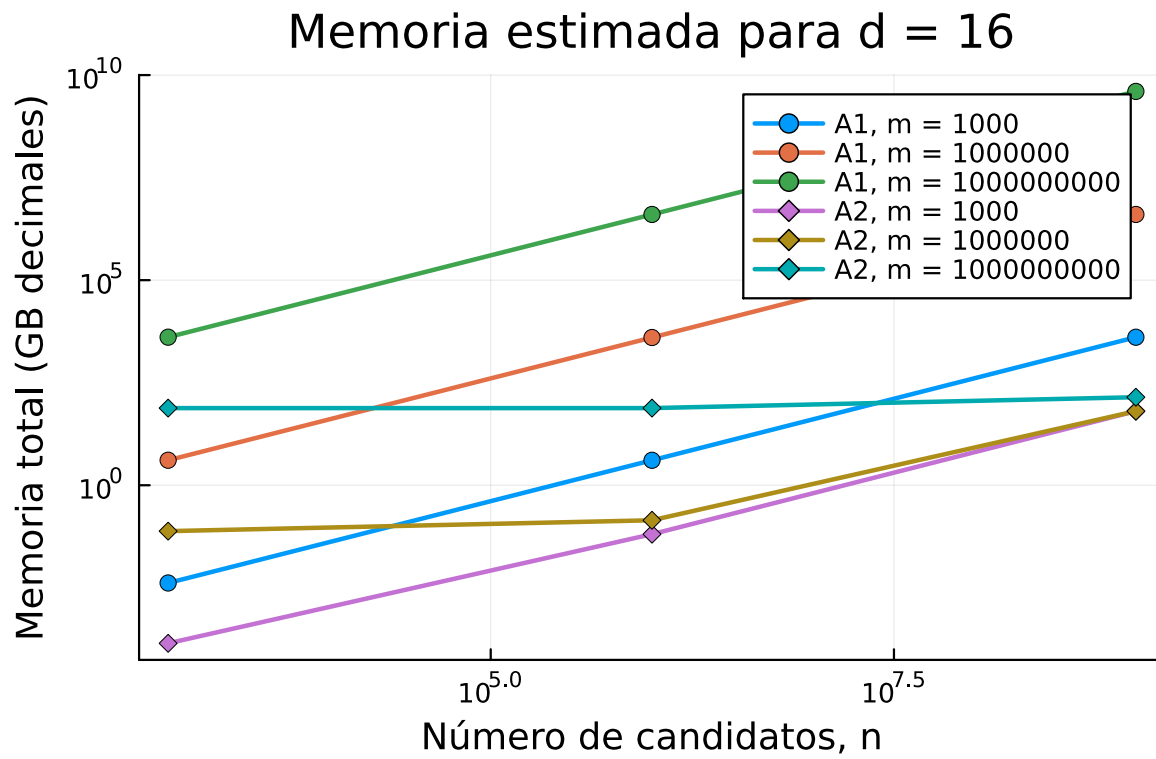
figura_memoria_8 = graficar_memoria(8)
figura_memoria_8

```



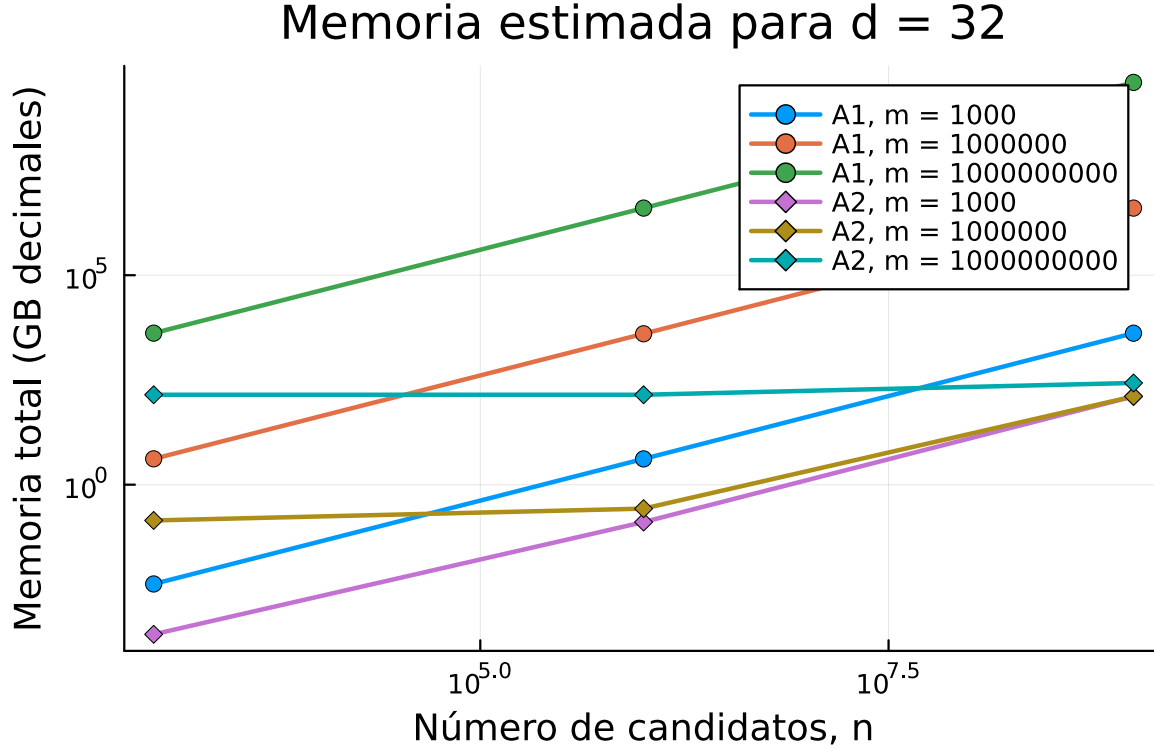
1.3.7 Gráfica para $d = 16$

```
figura_memoria_16 = graficar_memoria(16)
figura_memoria_16
```



1.3.8 Gráfica para $d = 32$

```
figura_memoria_32 = graficar_memoria(32)
figura_memoria_32
```



1.3.8.1 Discusión

Las tres gráficas muestran la memoria total estimada para $d=8$, $d=16$ y $d=32$. En cada gráfica, cada línea corresponde a A1 o A2 para uno de los tres tamaños de m ; el eje horizontal representa n . Además la memoria se expresa en GB decimales. La tabla y las gráficas cubren las 27 combinaciones indicadas en el enunciado.

La diferencia principal entre los algoritmos es la matriz temporal de productos. A1 almacena los mn productos punto en una matriz Float32, por lo que requiere $4mn$ bytes adicionales. A2 calcula cada producto punto y conserva solo el máximo actual, además de los arreglos de salida. **Así, para todas las combinaciones evaluadas, la estimación de A2 es menor que la de A1.** El acceso por columnas con @view evita copiar las columnas procesadas.

En A1, el tamaño de la matriz temporal depende de m y n , pero no de d ; por eso, cuando mn es grande, esa matriz domina ampliamente la estimación. En A2 no existe ese término temporal y el costo de memoria depende principalmente de las matrices de entrada y del tamaño m de las salidas.

En la combinación $m = n = 10^9$ y $d = 32$, A1 estima aproximadamente 4.0×10^9 GB decimales, mientras que A2 estima 268 GB, pero recordando que esto se calcula a partir de las dimensiones y de sizeof.

Para la corrida medida, con $m = 1000$, $n = 1000000$ y $d = 16$, la memoria total estimada es 4.064076 GB para A1 y 0.064076 GB para A2, donde la matriz temporal de A1 representa 4 GB de la primera cifra, es decir, esta matriz temporal abarca todo el espacio para este procesamiento.

1.3.9 Reporte los costos en tiempo real para $m = 10^3$, $n = 10^6$ y $d = 16$ para cada A1 y A2.

Para comparar el rendimiento de ambos algoritmos se utiliza el paquete BenchmarkTools.jl, que permite realizar mediciones del tiempo de ejecución y de las asignaciones de memoria (JuliaCI, s. f.).

```
d = 16
m = 10^3
n = 10^6

datos_exp = crear_datos(d, m, n)
Q = datos_exp.Q
X = datos_exp.X

# Se calculan los dots.
nns_A2, dots_A2 = top1_A2(Q, X)

trial_A1 = @benchmark top1_A1($Q, $X) samples = 5 seconds = 5 evals = 1
trial_A2 = @benchmark top1_A2($Q, $X) samples = 5 seconds = 5 evals = 1

tabla_tiempos = DataFrame(
    algoritmo = ["A1", "A2"],
    muestras = [length(trial_A1), length(trial_A2)],
    tiempo_mediano_segundos = [median(trial_A1).time / 10^9, median(trial_A2).time / 10^9],
    memoria_asignada_bytes = [median(trial_A1).memory, median(trial_A2).memory],
    asignaciones = [median(trial_A1).allocs, median(trial_A2).allocs]
)

println("Procesador: ", Sys.CPU_NAME, " (", Sys.CPU_THREADS, " hilos)")
display(tabla_tiempos)
```

Procesador: alderlake (16 hilos)

	algoritmo	muestras	tiempo_mediano_segundos	memoria_asignada_bytes	asignaciones
	String	Int64	Float64	Int64	Int64
1	A1	1	11.0471	4000012341	9
2	A2	1	8.65433	12222	6

Tenemos que A1 tardó 11.1797 segundos y A2 tardó 7.74129 segundos donde **Benchmark** registró una sola muestra para cada algoritmo.

El **benchmark** asignó 4,000,012,325 bytes durante la llamada a A1 y 12,238 bytes durante la llamada a A2. Q y X se generaron antes del **benchmark** y se pasaron como entradas ya existentes; por eso, **esta columna no incluye la memoria de esas matrices**. Esto explica por qué las asignaciones reportadas no son iguales a la memoria total estimada en la sección anterior.

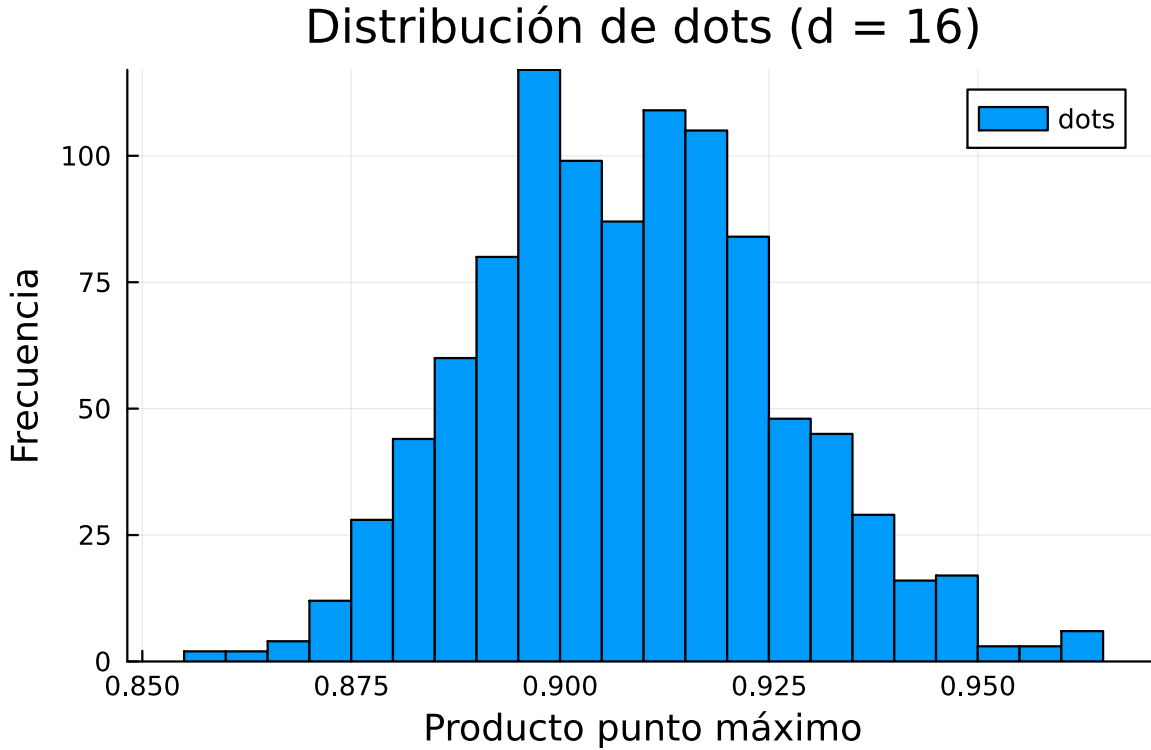
1.3.10 Grafique la distribución del arreglo *dots* para la corrida previa, i.e.,

$$m = 10^3, n = 10^6 \text{ y } d = 16.$$

Para interpretar el histograma, cada `dots[i]` es el máximo de n productos punto para la consulta i . Como las columnas tienen norma uno, sus productos punto se encuentran entre -1 y 1 . El histograma muestra cómo se distribuyen

```
grafica_dots = histogram(
    dots_A2,
    bins = 40,
    xlabel = "Producto punto máximo",
    ylabel = "Frecuencia",
    title = "Distribución de dots (d = 16)",
    label = "dots"
)

grafica_dots
```



En esta corrida, los valores de dots que aparecen en el histograma se encuentran aproximadamente entre 0.85 y 0.96, y la mayor concentración visual está alrededor de 0.90 a 0.92. Esto indica que, entre un millón de candidatos, el máximo producto punto de cada consulta fue positivo y cercano a 1.

1.4 Conclusiones

En esta actividad se resolvió el problema de producto punto máximo con dos algoritmos. A1 calcula primero la matriz completa $Q^T X$ y después busca el máximo de cada fila. A2 calcula los productos punto de una consulta a la vez y conserva únicamente el mejor índice y su valor.

La prueba pequeña confirma que ambos procedimientos respetan la definición del problema: nns guarda el índice del candidato con el puntaje más alto y dots guarda ese puntaje. En esa prueba los resultados coinciden exactamente.

Los dos algoritmos recorren las m consultas y comparan sus productos punto con los n candidatos; cada producto utiliza d coordenadas. Por ello, el trabajo de cálculo crece en el orden de $d \times m \times n$. La diferencia principal está en el espacio adicional: A1 almacena la matriz de $m \times n$ puntajes, que ocupa $4 \times m \times n$ bytes con Float32, mientras que A2 procesa una columna a la vez y mantiene el máximo actual. Ambos algoritmos necesitan las matrices de entrada y los arreglos nns y dots, pero A2 evita la matriz temporal. Por ejemplo, para

$m = n = 10^9$ y $d = 32$, la estimación de A1 es aproximadamente 4.0×10^9 GB decimales y la de A2 es 268 GB, son estimaciones.

En la única corrida cronometrada, con $m = 1000$, $n = 1000000$ y $d = 16$, A1 tardó 11.1797 segundos y A2 tardó 7.74129 segundos. A2 también registró menos memoria asignada dentro de la función: 12,238 bytes, frente a \$4,000,012,325 \$bytes de A1. Estos resultados muestran que A2 fue más rápido y requirió muchas menos asignaciones en esa corrida.

El histograma complementa la medición con la distribución de los 1000 máximos obtenidos para el mismo caso. Como las columnas están normalizadas, los productos punto se encuentran entre -1 y 1 ; en la corrida mostrada, los máximos se concentraron aproximadamente entre 0.90 y 0.92, dentro de un intervalo observado cercano a 0.85–0.96. Esto describe únicamente los datos generados para esa combinación.

En conjunto, A1 ofrece la formulación explícita mediante una matriz de puntajes, pero su memoria temporal fue la problemática; A2 evita almacenar esos puntajes y presenta una ventaja clara de memoria para los tamaños analizados.

1.5 Referencias

Téllez, E. S. (s. f.-a). Estructuras de datos elementales (capítulo 3). Análisis de algoritmos con Julia. <https://sadit.github.io/ALGO-IR/cap3-estructuras.html>

Téllez, E. S. (s. f.-b). Introducción al análisis de algoritmos con Julia (capítulo 2). Análisis de algoritmos con Julia. <https://sadit.github.io/ALGO-IR/cap2-analisis.html>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (2nd ed.). MIT Press.

Sipser, M. (2013). Introduction to the theory of computation (3rd ed.). Cengage Learning.

JuliaLang. (s. f.). Collections and data structures. The Julia Language. <https://docs.julialang.org/en/v1/base/collections/>

JuliaCI. (s. f.). BenchmarkTools.jl: Manual. <https://juliaci.github.io/BenchmarkTools.jl/stable/manual/>

Julia contributors. (s. f.). Random numbers. The Julia 1.12 documentation. <https://docs.julialang.org/en/v1/stdlib/Random/>

Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4.^a ed.). Johns Hopkins University Press.